

MCP as the connector pattern

Claude Certified Developer, Foundation (CCDV-F) · UST masterclass

EN

Session 4 of 12 · English notes

The last third of session 4 introduced **MCP (Model Context Protocol)** as the way Claude clients attach third-party systems. The live example was not an abstract protocol lecture: enable the Figma MCP server, authenticate, inspect the tool list, pass a Figma URL into a frontend skill, and let Claude Code generate a React/Vite/TypeScript UI from design context. Building your own MCP servers is promised for API modules; here you need the connector pattern.

PROTOCOL VERSUS THE FIGMA DEMO

Marketplace server names, OAuth screens, and the exact tool list (get screenshot, get design context, get metadata...) will change. The durable ideas: MCP exposes *tools* the model can plan over; you consent per invocation; results land as temporary files in a sandbox; you still write skills and CLAUDE.md so generation follows your stack. Current docs: [MCP](#) and [tool use](#).

Why MCP exists in this course

A model only sees what you put in the prompt, attach as files, or fetch via tools. Skills and CLAUDE.md cannot log into Figma. MCP is the standardised way to say: this server offers operations (tools); Claude may call them; the host (Claude Code / Claude.ai / Cowork) brokers auth and sandboxing. The trainer's one-line version: *get third-party details using a protocol called MCP* — Figma today, internal data later when you implement servers yourselves.

- Not mandatory for every frontend: you could paste HTML from Cowork Design straight into Code. The Figma hop was explicitly “I want to show MCP.”
- Special permissions: MCP tools must be enabled/connected; a disconnected server will not appear in the plan.
- Same Claude account and Figma account need to complete the OAuth/consent so a token can be issued to the client.

Connecting a server (Figma in the lab)

1. In Claude Code (or the Claude MCP settings the UI then showed), open MCP servers and install/connect the Figma server.
2. Authenticate: the client opens a browser URL; you approve access (email, token). Use the profile that actually owns the Figma file.
3. Confirm status: connected. Re-run “MCP servers” (or the equivalent panel) if the UI still shows disconnected.

4. Open the tool list. In the recording: get screenshot, get design context, get motion context, get metadata, get diagrams, export video, code-content mappings, and similar. You do not memorise the list; you understand that *the model plans which tools to call*.
5. Copy the *specific file URL* from Figma. A generic “use my Figma” prompt can scan an entire account and waste tokens.

Figma-side, you may also create a token if the connector asks for it; the demo mostly used the OAuth path after logging into the correct Google profile. Share the Figma file if the Claude identity is not already a collaborator.

Frontend skill + memory + MCP in one prompt

Before the generate command, the trainer added a **frontend skill** (senior frontend engineer; React, TypeScript, Vite; responsive, accessible, reusable components; error handling; routing; optional Next.js if that is the house stack) and a frontend CLAUDE .md (task assignment, priorities, “honour this OpenAPI,” UI principles). Then a *new session*, slash the frontend skill, and a prompt of the form: generate the frontend from the Figma design at this URL.

Claude’s first moves are MCP, not JSX: “do I have a Figma server? which tools? get metadata, get design context...” Each call is a function call. You will see consent prompts (“do you want to proceed?”). Approve with the same seriousness you give shell commands.

Where tool results live: sandbox, not the chat forever

A learner asked whether the generated text dump of Figma metadata was a temporary file. Yes. MCP/tool results are stored as temp artefacts: metadata often as .txt, JSON APIs as JSON, in a **sandbox** on the local/ Claude workspace. The previous session had already introduced that sandbox; this session reused it. Think of it as an isolated environment (the trainer compared it to a Docker-like OS where Node or Python can be installed — analogy, not “MCP runs Docker”). The LLM injects lists of files/folders; IO in the sandbox actually writes the tree (code-interpreter style). After the initial MCP burst, generation uses that cached design context rather than re-fetching every component.

You see in the UI	What it usually is	What you should do
Consent to get_metadata / get_design_context	Tool call to the MCP server	Read the tool name; deny if the URL is wrong.
Temporary txt/json in the sandbox	Tool result payload	Do not commit secrets; treat as ephemeral.
Scaffold (Vite, React, Tailwind, src/, package.json)	Local IO after planning	Review gitignore; do not push node_modules.
Mismatch warnings (UI screen vs missing API)	Model comparing design to OpenAPI	Skip, add an endpoint, or drop the screen — do not ignore silently in production.

Token and usage notes from the live run

End-to-end frontend generation in the recording sat around 10–12% of the trainer’s usage meter for that app size. Large products should be chunked. Verification (opening a browser, checking UI) can add ten or more minutes — that is why the session stopped mid-generation and resumed on 12 August. Prefer Mermaid-like structured design context over shipping giant screenshots when you can; MCP “get design context” is already an attempt to give structure instead of pixels only.

What MCP is not (yet) in this chapter

- Not a replacement for CLAUDE.md or skills — those still constrain stack and quality after context arrives.
- Not automatic write access to Figma; it is read/design-context for generation unless you enable other tools.
- Not your custom UST data source. Internal MCP servers come when you define tools on the API side.
- Not a reason to skip OpenAPI: the model still noticed missing user-management endpoints versus a user-list screen.

Official documentation

- **MCP**

<https://docs.claude.com/en/docs/agents-and-tools/mcp>

Connector pattern: servers, tools, and how Claude hosts them.

- **Tool use overview**

<https://docs.claude.com/en/docs/agents-and-tools/tool-use/overview>

Function-calling shape that MCP tools sit on.

- **Claude Code overview**

<https://code.claude.com/docs/en/overview>

Where MCP servers are enabled in the coding client.

- **Claude Docs home**

<https://docs.claude.com/en/home>

Jump to current product pages if the Figma connector UI has moved.

Study notes distilled from the UST CCD Foundation masterclass. Prefer Anthropic’s current docs over session anecdotes. Not official exam material.